Tema 1 - Tecnologías móviles. Fundamentos de Android - FILTRADO (lo que entra)

Programación Multimedia y Dispositivos Móviles - 2º DAM - Curso 26/27

1. Introducción

POR ENCIMA

Idea general: el smartphone + redes móviles han unificado en un solo dispositivo multitud de tareas. Antes de programar hay que conocer tecnologías, dispositivos y condicionantes.

1.1. Características de los dispositivos móviles

POR ENCIMA

Definición: aparato con capacidad de procesado y almacenamiento, pequeño, portable, con autonomía de batería y conexión permanente/semipermanente a red, con variadas funciones. 4 bloques:

1. **Capacidad de procesado**: cálculo + almacenamiento -> se les llama microordenadores.
2. **Tamaño**: portabilidad extrema, acompaña al usuario siempre.
3. **Movilidad**: sin cables para alimentación/comunicación. Pilares: pequeño tamaño + batería + comunicación inalámbrica.
4. **Conectividad**: redes móviles + redes de datos -> nuevo paradigma.

- Extra: SO, diseño, ergonomía, pantalla, gadgets/sensores.

1.2. Tipos de dispositivos móviles (por encima - ver los tipos)

- **Teléfonos móviles**: solo voz (+ SMS, cámara, agenda en algunos).
- **Handhelds**: agenda/organizador, pantalla plegable sobre teclado, apps científicas/financieras.
- **Netbooks**: 1999 Psion, hasta 2007. Compactos, baratos, ofimática + internet.
- **E-Book Readers**: leer documentos, ligeros, gran almacenaje, bajo consumo, tinta electrónica e-Ink.
- **Tablets**: entre móvil y PC, ligereza + autonomía + táctil. Popularización por iPad Apple.
- **PDA (Personal Digital Assistant)**: organizador digital, lápiz stylus, decayeron con smartphones.
- **Smartphones**: teléfono + prestaciones de PC + sensores (cámara, GPS...), pantalla alta calidad, conectividad múltiple, gran procesado/almacenaje.

1.3. Limitaciones de los dispositivos móviles

POR ENCIMA

- No prototipar solo para gama alta: si va bien en gama baja, irá mejor en alta. Optimizar CPU.
- Diversidad de **pantalla** (tamaños/resoluciones) y **sensores** (puede impedir instalar).
- Diferencias de **plataforma/versiones**: no hay retrocompatibilidad garantizada. Cuidado con librerías de terceros.
- **Red**: ancho de banda limitado, desconexiones por cobertura -> no exigir conexión constante.

- Probar en el máximo de dispositivos reales; si no, usar **emuladores (AVD)**.

---

2. Sistemas operativos móviles

SI

Modelo de capas (de hardware hacia arriba):

- **Núcleo o kernel**: acceso hardware vía drivers, gestión procesos, memoria, sistema de archivos.
- **Middleware**: módulos transparentes al usuario (mensajería, códecs multimedia, intérpretes web, gestión dispositivo, seguridad).
- **Entorno de ejecución**: gestor de apps e interfaces abiertas para desarrolladores.
- **Interfaces de usuario**: presentación visual (botones, pantallas, listas) + marco de interacción.
- **Aplicaciones nativas**: propias de cada modelo/fabricante.

2.1. Android

SI

- Lanzado **2007**, líder del mercado. Desarrollado por **Open Handset Alliance (OHA, 78 compañías lideradas por Google)**.
- Carácter **abierto**: kernel bajo **GNU GPLv2**, resto bajo **Apache v2**.
- Arquitectura en capas:
- a) **Kernel Linux**: base del runtime ART (hilos, memoria bajo nivel).
- b) **HAL (Hardware Abstraction Layer)**: enlaza hardware con framework. Una librería por componente (cámara, bluetooth...).
- c) **Runtime (ART)**: cada app corre en su proceso con su instancia ART. Ejecuta **archivos DEX** (bytecode Android, mínimo consumo de memoria), varias VM en poca memoria.
- d) **Bibliotecas nativas C/C++**: muchos servicios dependen de código nativo, expuestas vía API.
- e) **Framework Java API**: todo el SO accesible vía API Java.
- f) **Apps del sistema**: email, SMS, calendario, navegador.

2.2. iOS - sí

- De Apple, antes **iPhoneOS**, derivado de **MacOS X**. Fuerte: integración hardware-software + pantalla multitáctil (manipulación directa). Arquitectura en capas (arriba = más alto nivel para desarrollo):
- a) **Cocoa Touch** (superior, visible): UIKit (interfaz) + Foundation (objetos, servicios SO).
- b) **Media Services**: audio, gráficos, multimedia.
- c) **Core Services**: base de datos, red.
- d) **Core OS**: núcleo bajo nivel (memoria, seguridad, drivers).

2.3. Otros SO

FUERA

> **FUERA -** Omitido: SymbianOS, BlackBerryOS, Windows Phone / 10 Mobile, Ubuntu Touch. No estudiar.

3. Nociones previas al desarrollo

SELECTIVO

3.1. Desarrollo nativo

HAY QUE SABÉRSELO

- Cada plataforma usa lenguajes/herramientas propios.
- Ventaja: **flexibilidad total, adaptación total, máximo rendimiento**.
- Desventaja: para multi-entorno hay que dominar varios lenguajes/herramientas -> más tiempo. Los programadores suelen especializarse en una plataforma.

3.1.1. Desarrollo multiplataforma compilado a nativo

SECUNDARIO

- Idea: un solo lenguaje para todas las plataformas, con pequeñas adaptaciones. Se compila a **código nativo** -> mismo rendimiento que nativo + acceso directo a API y controles UI nativos.
- Ejemplo: **Xamarin (C# + .NET, reutiliza código menos la interfaz)**, hoy descontinuado en favor de **.NET MAUI**.

3.2. IDEs

3.2.1. iOS

POR ENCIMA

- Lenguaje: **Swift** (para Cocoa/Cocoa Touch, mejor que Objective-C: menos tolerante a errores, sintaxis más simple).
- IDE: **Xcode** (solo MacOS) + Interface Builder + depuración.

3.2.2. Android

SI

- Lenguaje: **Kotlin** (oficial/preferente, interoperabilidad 100% con Java). Moderno, conciso, null-safety, corrutinas para asincronía.
- UI moderna: **Jetpack Compose** (declarativo en Kotlin). **XML** sigue para recursos (strings, colores, dimensiones, vectores) y mantenimiento de layouts clásicos.
- IDE: **Android Studio** (Google, sobre IntelliJ IDEA de JetBrains). Compilación con **Gradle**, emulación con **AVD**.

4. Desarrollo de aplicaciones Android

IMPORTANTE

4.1. Componentes de una aplicación

IMPORTANTE

4 tipos, cada uno con fin y ciclo de vida propios:

- **Activity**: pantalla con UI, gestiona interacción con usuario. Una app puede tener varias que colaboran.
- **Service**: corre en fondo, operaciones largas o remotas. Sin UI. Una Activity puede iniciarlo o enlazarse.
- **Content Provider**: gestiona datos para compartir. Otras apps pueden consultar/modificar (si se permite).
- **Broadcast Receiver**: responde a broadcasts del sistema. Normalmente hace poco trabajo y enlaza con otros componentes.

4.2. Android Manifest

IMPORTANTE

- Archivo `AndroidManifest.xml`. El sistema lo lee para reconocer componentes.
- Todas las apps lo tienen; **todos los componentes deben declararse aquí**.
- Funciones: informar componentes, registrar **permisos**, **librerías**, necesidades **hardware/software**, **nivel API mínimo**. Escrito en lenguaje de marcas.
- Ejemplo típico: declara `MainActivity` como punto de entrada (`action MAIN` + `category LAUNCHER`), con `exported="true"`, icono, label, tema.

4.3. Recursos de una aplicación

SECUNDARIO

- Idea: datos que usa la app (imágenes, textos, estilos) + código = aplicación.
- **Predeterminados** (siempre) vs **alternativos** (según configuración: tablet, pantalla grande... -> responsividad).
- Gestionados en XML, con **id**, en subcarpetas de `res/` **(nombres predefinidos por Android):**
- `drawable`: bitmaps / escalables, formas y colores.
- `layout`: XML con elementos visuales de la interfaz.
- `animator`: animaciones (rotación, fading, movimiento, estiramiento), un XML por animación.
- `mipmap`: bitmaps, se usa para el **icono de la app**.
- `menu`: menús, submenús, barras de navegación.
- `values`: genérico (colores, dimensiones, strings, estilos).
- `xml`: XML de soporte (p. ej. configuración de búsqueda).
- `raw`: audio/vídeo raw (limitación de nombres: sin mayúsculas/números/especiales; si necesitas nombre real usa `assets/`).

4.4. La clase R

IMPORTANTE

- Contiene **variables estáticas** que identifican cada recurso. Android carga XML en memoria y mantiene `R` como referencia directa (cada atributo -> dirección de memoria de un recurso).
- En Kotlin: clases estáticas internas por tipo (`R.string`, `R.layout`, `R.drawable`, `R.id`...). No guardan los datos, guardan **Int únicos** (claves en la tabla de recursos empaquetada).
- Uso en ejecución: `getString(R.string.app_name)`, `painterResource(id = R.drawable.icono)` -> el framework carga solo lo pedido bajo demanda.

4.5. Actividades

SUPER IMPORTANTE

- **Activity = pantalla**. Hereda de clase `Activity`. En general: tantas Activities como pantallas.
- Varias Activities vinculadas con secuencia según lógica/flujo y decisiones del usuario. Desde main se navega de una a otra.
- Android recuerda la secuencia en una pila: **Back Stack**. Al iniciar una actividad nueva, obtiene el foco y se empuja arriba; la anterior queda detenida en la pila. Retroceder = desapilar en orden inverso.
- **Tarea (Task)**: conjunto de actividades con relación lógica en la pila. Puede ir a segundo plano (usuario abre otra tarea o va a Inicio): todo se para pero el Back Stack se conserva. Al volver a primer plano, se retoma donde se dejó.

4.5.1. Ciclo de vida de una actividad

100%

Estados:

- **Nueva**: sin memoria, no interactuable.
- **Detenida**: en memoria, no visible, no accesible.
- **Pausada**: visible pero no interactuable.
- **Activa**: en primer plano, interactuable.
- **Eliminada**: borrada de memoria.

Callbacks (llamados por Android):

- `onCreate()`: primera invocación. Crea vistas/UI, reserva memoria, `findViewById()`, consultas iniciales, guarda variables entorno.
- `onStart()`: tras `onCreate()`. Visible pero aún no interactuable. Instrucciones asociadas a UI.
- `onResume()`: pasa a interactuable. Tras `onStart()`, antes de `onPause()`. Activar sensores/cámara, animaciones, actualizar info.
- `onPause()`: pasa a segundo plano (se va a destruir o la sustituye otra). Parcialmente visible. Detener procesos/animaciones. Puede volver a `onResume()` o ir a `onStop()`.
- `onStop()`: ya no visible (otra en primer plano). Pausa animaciones, detiene servicios (GPS), minimiza recursos, conserva datos en memoria. Reactivar con `onRestart()` o eliminar con `onDestroy()`.
- `onRestart()`: actividad parada vuelve a primer plano. Siempre le sigue `onStart()`.
- `onDestroy()`: destrucción definitiva, fuera de memoria. Limpiar/liberar recursos.

Ciclos:

- **Completo**: `onCreate()` ->`onDestroy()`.
- **Visible**: `onStart()` ->`onStop()` (usuario la ve en pantalla).
- **Primer plano**: `onResume()` ->`onPause()` (tiene el foco, por encima de todas).

4.6. Intents y filtros

IMPORTANTE

- **Intent** = comunicación entre componentes (misma o distintas apps). Permite iniciar app/servicio, entregar mensaje, enviar/recibir info entre actividades/servicios, difundir eventos a receptores. Permite reutilizar (p. ej. abrir cámara sin programarla).
- Dos tipos: explícitos e implícitos.

4.6.1. Intent explícito

- Se conoce el **componente destino exacto** (normalmente del propio paquete). Para navegación interna o invocar servicios, sin que otra app intercepte.
- Transporta datos con **extras / Bundle** vía `putExtra()` (+ extensiones Kotlin): primitivos, strings, serializables.

```
// Origen (MainActivity.kt)
val intentExplicito = Intent(this, UsuarioActivity::class.java).apply {
    putExtra("EXTRA_USUARIO_ID", 777)
    putExtra("EXTRA_NOMBRE", "Alejandro Monfort")
}
startActivity(intentExplicito)

// Destino (UsuarioActivity.kt)
val idUsuario = intent.getIntExtra("EXTRA_USUARIO_ID", -1)
val nombreUsuario = intent.getStringExtra("EXTRA_NOMBRE")
```

4.6.2. Intent implícito

- No define componente, declara **acción abstracta** (ver web, compartir, llamar, mapa) + datos. Android busca `<intent-filter>` en todas las apps instaladas; si hay varias, muestra selector.
- Reutiliza ecosistema, desacopla responsabilidades. Para obtener resultado de vuelta (elegir archivo, foto): **Activity Result API** con `registerForActivityResult()` (seguro aunque el sistema destruya la actividad en fondo).

```
val sendIntent = Intent(Intent.ACTION_SEND).apply {
    type = "text/plain"
    putExtra(Intent.EXTRA_TEXT, "Solicitud de texto")
}
val chooserIntent = Intent.createChooser(sendIntent, "Compartir mensaje mediante:")
if (sendIntent.resolveActivity(packageManager) != null) {
    startActivity(chooserIntent)
}
```

4.6.3. Elementos de un Intent

CAE EN PRÁCTICA

Memorizar campos del constructor + ejemplo:

- **Nombre del Componente** (`ComponentName`): clase + paquete destino exacto. Su presencia = explícito, su ausencia = implícito.
- **Acción**: string abstracto de operación/evento. Constantes: `ACTION_VIEW`, `ACTION_SEND`, `ACTION_DIAL`.
- **Datos y Tipo MIME**: `data` = URI (`android.net.Uri`: web, teléfono, geo) + `type` = MIME (`image/png`, `text/plain`). Van juntos.

- **Categoría**: string complementario que restringe entorno/componente. `addCategory()`.
- **Datos extra**: clave-valor en **Bundle** para la lógica del destino. `putExtra()` sobrecargado.
- **Indicadores (flags)**: enteros binarios de `Intent` que instruyen al SO (p. ej. gestión de pila de tareas). `flags = ...` o `addFlags()`.

```
// Explícito
val nombreIntent = Intent(contextoOrigen, ClaseDestino::class.java).apply {
    putExtra("CLAVE_DATO_1", valorDato1)
    putExtra("CLAVE_DATO_2", valorDato2)
    addFlags(Intent.FLAG_ACTIVIDAD_DESEADA)
}
startActivity(nombreIntent)

// Implícito
val nombreIntent2 = Intent(Intent.ACTION_NOMBRE_ACCION).apply {
    data = Uri.parse("esquema:recurso")
    type = "tipo/mime"
    putExtra(Intent.EXTRA_TEXTO_OPCIONAL, valor)
    addCategory(Intent.CATEGORY_NOMBRE_CATEGORIA)
}
startActivity(nombreIntent2)
```